

PROGRAMACIÓN AVANZADA

Interfaz de Usuario
(GUI) Swing – JTable



Ing. Matias Macrino

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Componente JTable.

Es uno de los componentes mas interesantes que aporta Swing. Su utilización es generalmente para mostrar conjuntos de datos en una cuadrícula bidimensional (filas y columnas).

Un JTable puede obtener los datos a visualizar de una colección o array de objetos, pero también de cualquier otra fuente de datos cuyos criterios de obtención de información estén definidos en una clase que implemente la interfaz **TableModel**.

No contiene datos ni almacena los mismos; simplemente es una vista de ellos.

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Componente JTable - Creación.

Sus constructores pueden incluir los objetos donde residen los datos y los objetos con los títulos correspondientes. Dicho componente tiene los siguientes constructores:

- > `JTable();`
- > `JTable(Object[][] rowData, Object[] columnName);`
- > `Jtable(TableModel dataModel, TableColumnModel columnModel);`

Su constructor por defecto, construye una tabla predeterminada:

- > basada en un modelo por defecto, con nombres por defecto y selección por defecto
- > editable
- > ajusta el ancho de las columnas al tamaño de los datos
- > ajusta el tamaño horizontal para evitar la barra de scroll

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Componente *JTable* - Creación.

Constructor ***JTable(Object[][] rowData, Object[] columnName)***: dicho constructor, construye una tabla con los datos del array bidireccional ***rowData*** y los rótulos establecidos en ***columnName***, con selección por defecto.

Ejemplo:

```
String[] columnNames = {"Nombre", "Apellido", "Pasatiempo", "Años de Practica", "Soltero/a"};
Object[][] data = { {"Maria", "Campione", "Esquiar", new Integer(5), new Boolean(false)},
                    {"Lucas", "Hum", "Patinar", new Integer(3), new Boolean(true)},
                    {"Carla", "Gonzalez", "Escalar", new Integer(2), new Boolean(false)},
                    {"Marcos", "Perez", "Correr", new Integer(7), new Boolean(true)},
                    {"Angela", "Esposito", "Nadar", new Integer(4), new Boolean(false)} };

JTable table = new JTable(data, columnNames);
```

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Componente **JTable** - Creación.

Para asegurar la visualización de los títulos, el componente **JTable** debe agregarse a un contenedor del tipo **JScrollPane** o bien, explícitamente al contenedor del tipo **JFrame**

Ejemplo:

-> Con **JScrollPane**:

```
tabla.setPreferredScrollableViewportSize(new Dimension(500, 70));    //tamaño de la vista de JTable
JScrollPane scrollPane = new JScrollPane(tabla);
add(scrollPane);                                                       // no es necesario agregar Jtable
```

-> Sin **JScrollPane**:

```
add(tabla.getTableHeader());      // columnas agregadas al JFrame
add(tabla);                       // agrego JTable al JFrame
```

* Otra alternativa en lugar de usar array, se pueden utilizar objetos de la clase **Vector**.

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

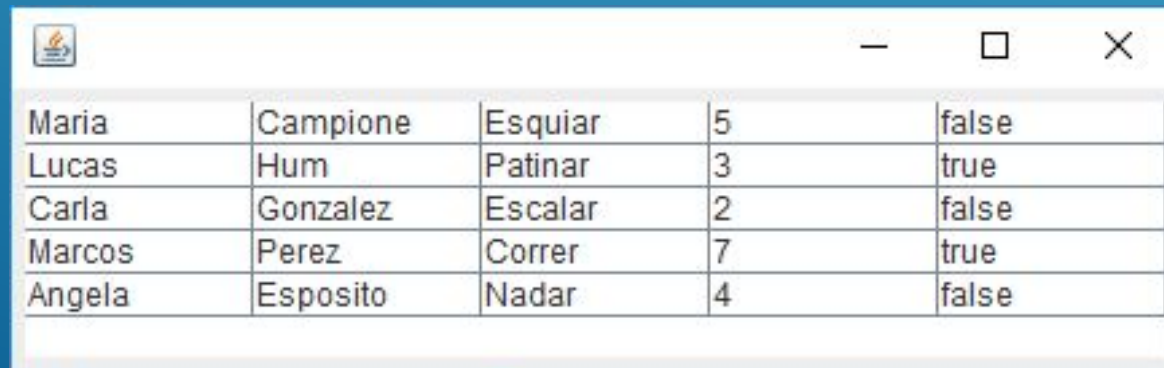
Componente *JTable* - Creación.

-> Sin *JScrollPane*:

```
String[] columnNames = {"Nombre", "Apellido", "Pasatiempo", "Años de Practica", "Soltero/a"};
Object[][] data = { {"Maria", "Campione", "Esquiar", new Integer(5), new Boolean(false)},
                    {"Lucas", "Hum", "Patinar", new Integer(3), new Boolean(true)},
                    {"Carla", "Gonzalez", "Escalar", new Integer(2), new Boolean(false)},
                    {"Marcos", "Perez", "Correr", new Integer(7), new Boolean(true)},
                    {"Angela", "Esposito", "Nadar", new Integer(4), new Boolean(false)} };

JTable table = new JTable(data, columnNames);

add(table.getTableHeader());
add(table);
```



Maria	Campione	Esquiar	5	false
Lucas	Hum	Patinar	3	true
Carla	Gonzalez	Escalar	2	false
Marcos	Perez	Correr	7	true
Angela	Esposito	Nadar	4	false

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

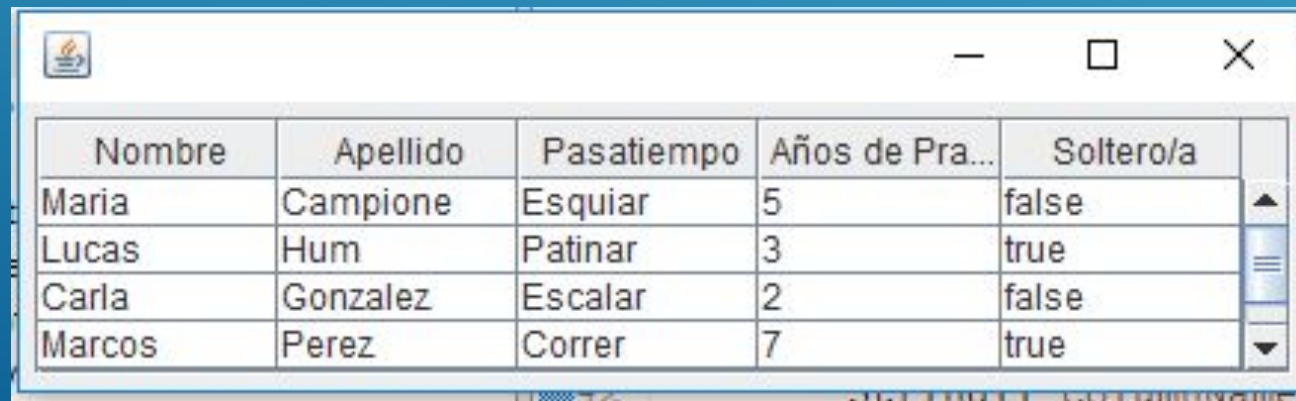
Componente *JTable* - Creación.

-> Con *JScrollPane*:

```
String[] columnNames = {"Nombre", "Apellido", "Pasatiempo", "Años de Practica", "Soltero/a"};
Object[][] data = { {"Maria", "Campione", "Esquiar", new Integer(5), new Boolean(false)},
                    {"Lucas", "Hum", "Patinar", new Integer(3), new Boolean(true)},
                    {"Carla", "Gonzalez", "Escalar", new Integer(2), new Boolean(false)},
                    {"Marcos", "Perez", "Correr", new Integer(7), new Boolean(true)},
                    {"Angela", "Esposito", "Nadar", new Integer(4), new Boolean(false)} };

JTable table = new JTable(data, columnNames);

JScrollPane scrollPane = new JScrollPane(table);
add(scrollPane);
```



The screenshot shows a standard Java Swing window with a title bar containing a maximize button, a close button, and a scroll icon. The window contains a JTable with the following data:

Nombre	Apellido	Pasatiempo	Años de Pra...	Soltero/a
Maria	Campione	Esquiar	5	false
Lucas	Hum	Patinar	3	true
Carla	Gonzalez	Escalar	2	false
Marcos	Perez	Correr	7	true

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Componente *JTable* - Creación.

- ▶ Este uso del componente genera algunas desventajas:
 - ❖ Los datos a mostrar deben estar registrados en un array.
 - ❖ Las celdas siempre son editables.
 - ❖ Los datos a mostrar siempre son tratados como ***String***.
- ▶ Estas desventajas se eliminan al implementar el componente ***JTable*** con un ***modelo de tabla***.
- ▶ ***Los modelos de tabla*** son objetos que implementan la interface ***TableModel***; logrando personalizaciones más específicas, mejor comportamiento y aprovechamiento de las facultades del componente ***JTable***.

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Componente JTable - Creación.

- ▶ La clase ***AbstractTableModel*** es la que implementa directamente a la interface ***TableModel***.
- ▶ De esa clase, se debe extender para utilizarla como modelo de tabla. Aunque, existe un modelo de tabla predeterminado que facilita el trabajo con tablas: la clase denominada ***DefaultTableModel***.
- ▶ Para crear un objeto de la clase ***DefaultTableModel***, podemos utilizar el constructor similar al objeto ***JTable***.

DefaultTableModel(Object[][] data, Object[] columnNames)

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Componente JTable - Creación.

▶ Ejemplo:

```
DefaultTableModel dtm=new DefaultTableModel(data, columnNames);
```

```
JTable tabla = new JTable (dtm);           //creo JTable basado en el modelo
```

▶ Las modificaciones de datos realizadas en el modelo de tabla, se reflejan en el componente.

▶ **Agregar una fila:**

```
Object newRow[]={ "Pepe","Grillo","Tenis",new Integer(5), new Boolean(false)};  
dtm.addRow(newRow);
```

▶ **Agregar una columna:**

```
String[] newColumn= { "Flan","Pastel","Helado","Barquillo","Manzana" ,"Pera"};  
dtm.addColumn("Postres", newColumn);
```

▶ **Modificar el valor de una celda:**

```
dtm.setValueAt("Juana", 1, 1);           // columna 1, fila 1
```

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

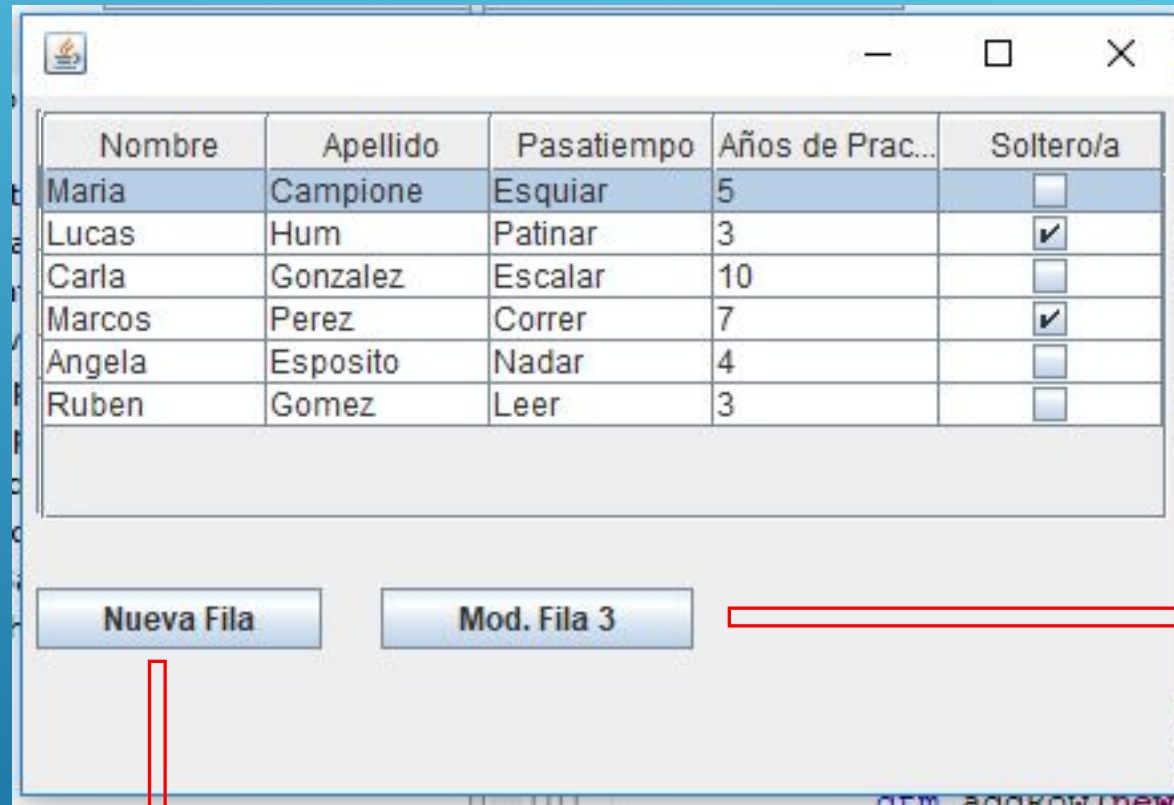
Componente *JTable* - Creación.

```
String[] columnNames = {"Nombre", "Apellido", "Pasatiempo", "Años de Practica", "Soltero/a"};
Object[][] data = { {"Maria", "Campione", "Esquiar", new Integer(5), new Boolean(false)},
                    {"Lucas", "Hum", "Patinar", new Integer(3), new Boolean(true)},
                    {"Carla", "Gonzalez", "Escalar", new Integer(2), new Boolean(false)},
                    {"Marcos", "Perez", "Correr", new Integer(7), new Boolean(true)},
                    {"Angela", "Esposito", "Nadar", new Integer(4), new Boolean(false)}  };

DefaultTableModel dtm;
dtm = new DefaultTableModel(data, columnNames);
JTable table = new JTable(dtm);
JScrollPane scrollPane = new JScrollPane(table);
add(scrollPane);
```

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Componente *JTable* - Creación.



```
dtm.setValueAt("10", 2, 3);
```

```
dtm.addRow(new Object[]{"Ruben", "Gomez", "Leer", new Integer(3), new Boolean(false)});
```

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Componente JTable - Creación.

► Ejemplo con **modelo de tablas dinámicas**:

```
Object[] data = new Object[5];
DefaultTableModel dtm= new DefaultTableModel();
JTable table = new JTable(dtm);
for(int column = 0; column < 5; column++){
    dtm.addColumn("Columna " + column);    // insertamos columnas
}
for(int row = 0; row < 10; row++) {
    for(int column = 0; column < 5; column++) {
        data[column] = "Celda " + row + "," + column;    // insertamos contenido de las columnas
    }
    dtm.addRow(data);
}
```

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Componente *JTable* - Creación.

- ▶ Las celdas siguen siendo editables y los datos tratados como *String*. Esto se debe al uso de modelo de tabla ***DefaultTableModel***.
- ▶ Aunque la utilización del mismo es funcional y proporciona facilidades para la implementación de un modelo de tabla, es mucho más recomendable, por cuestiones de desempeño, personalización y eficiencia, utilizar la clase ***AbstractTableModel***.
- ▶ Por ser una clase abstracta, se debe crear una clase propia que derive de la misma y re-escriba los métodos: ***getRowCount***, ***getColumnCount*** y ***getValueAt***.
- ▶ Si el modelo es editable también se debe sobrescribir: ***setValueAt*** y ***isCellEditable***.

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

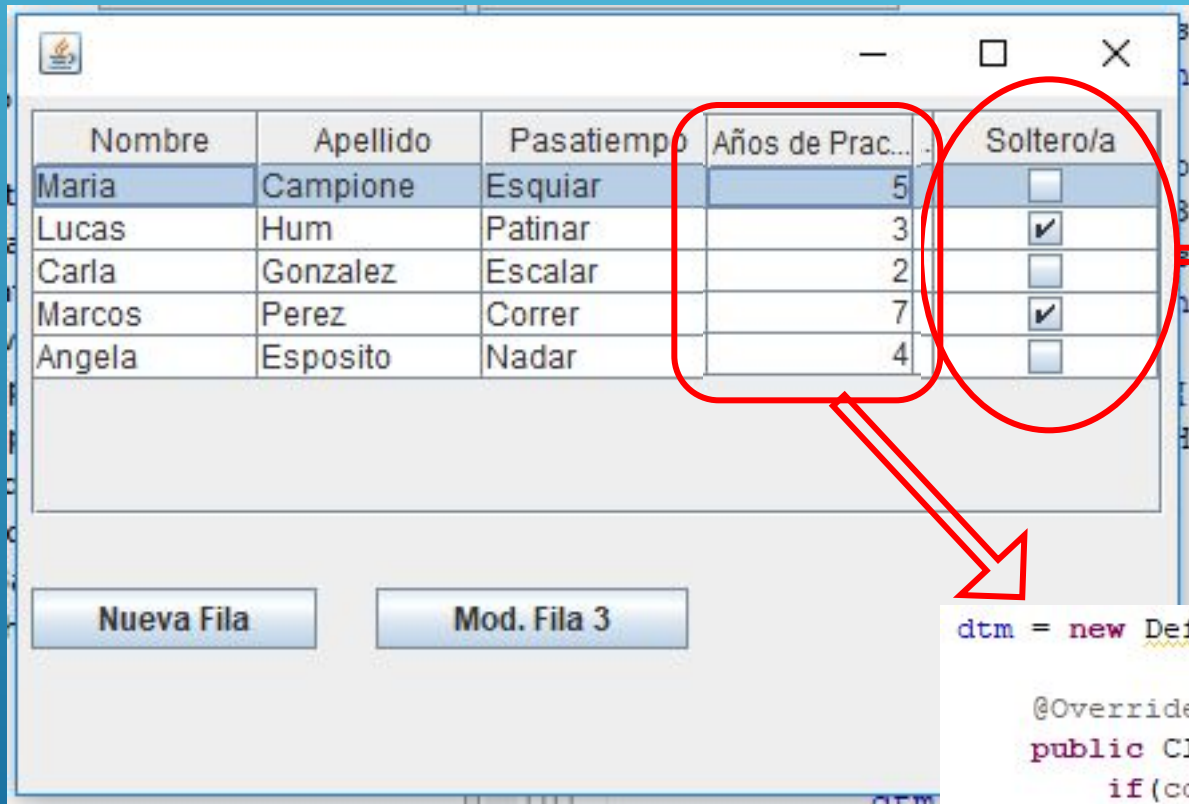
Componente *JTable* - Creación.

- Ejemplo con métodos a sobrescribir:

```
public class TableModelPropio extends AbstractTableModel {  
    final String[] columnNames = {...}  
    final Object[][] data = {...}  
    public int getRowCount(){ ...}  
    public int getColumnCount(){...}  
    public Object getValueAt(int row, int column){...}  
    // solo para JTable editables:  
    public void setValueAt(Object value, int row, int col) {...}  
    public boolean isCellEditable(int row, int col) {...}  
}
```

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Componente JTable - Creación.



```
dtm = new DefaultTableModel(data, columnNames) {  
  
    @Override  
    public Class<?> getColumnClass(int columnIndex) {  
        if(columnIndex == 4) {  
            return Boolean.class;  
        }  
        return Object.class;  
    }  
};
```

```
dtm = new DefaultTableModel(data, columnNames) {  
  
    @Override  
    public Class<?> getColumnClass(int columnIndex) {  
        if(columnIndex == 3) {  
            return Integer.class;  
        }  
        if(columnIndex == 4) {  
            return Boolean.class;  
        }  
        return Object.class;  
    }  
};
```

APLICACIONES BASADAS EN ENTORNO GRAFICO (SWING)

Componente *JTable* - Eventos

- Manejo de eventos se recomienda utilizar “**ListSelectionListener**”. El mismo se debe utilizar agregando el escuchador con el método “**addListSelectionListener**”. Antes cualquier cambio de estado del componente, se dispara el método “**valueChanged**”, el cual recibe un **ListSelectionEvent**, que nos permite conocer que sucedió.

```
tabla.addListSelectionListener(new ListSelectionListener() {  
    @Override  
    public void valueChanged(ListSelectionEvent e) {  
        [CODIGO.....]  
    }  
});
```